

# **Divide and Conquer**

## Quick Sort:



Randomly choose a pivot. Compare with it and put all smaller elements to its left and all larger elements to its right (equal not possible as indices are used to break tie)

Lexicographical (value then index) comparison!  $O(n)$  space

If you choose the median (best) every time, the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Time complexity:  $O(n \log n)$   
(best case)

Comparisons  
with pivot val.

## Intelligent Quick Sort:

Find median in linear time (based on select algo)  
and then apply comparisons!

Median  
finding



Time complexity:  $O(n \log n) \leftarrow \text{every iteration: } O(n) + O(n)$

If you choose the largest or smallest (worst) every time, the  
recurrence relation:

$$T(n) = T(n - 1) + O(n)$$

Time complexity:  $O(n^2)$   
(worst)

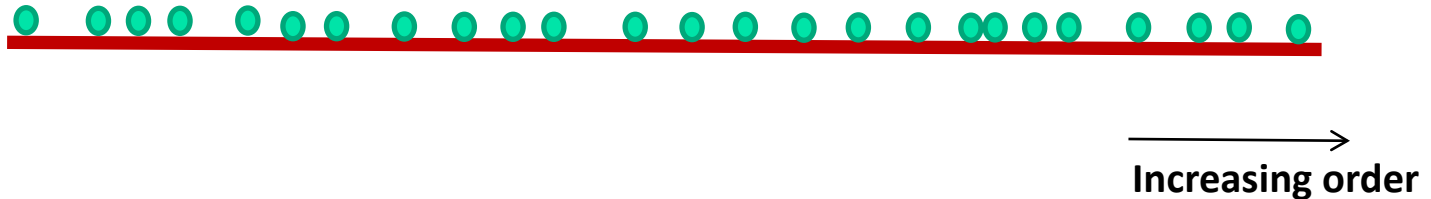
**Basic Quick Sort:** Just select the first or last element as the pivot.  
(why use random?)

## 'Randomized' Quick Sort:

Randomly choose the pivot.

We show- Expected time complexity is  $O(n \log n)$

$n$  numbers to be sorted in the number-line:



Expected runtime:

- Runtime proportional to the number of comparisons
- Any comparison between two particular elements (one pivot) will happen only once, as a pivot is not used in subsequent runs

Consider R.Vs

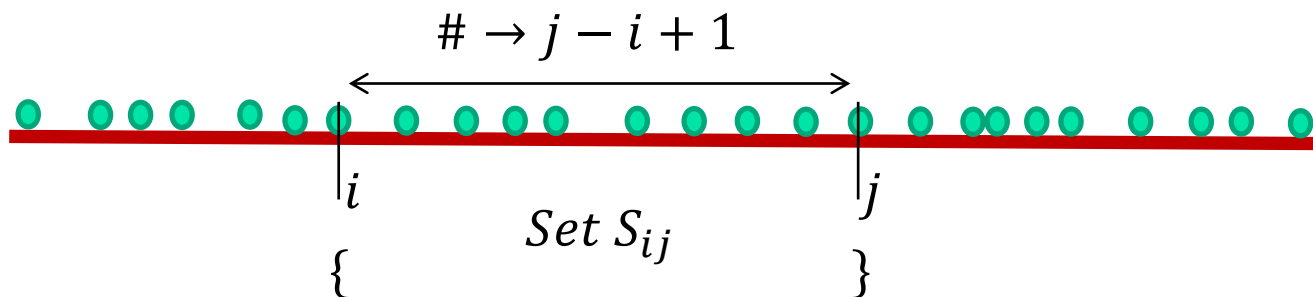
$$X_{ij} = \begin{cases} 1, & \text{if } i^{\text{th}} \text{ smallest element is ever compared to the } j^{\text{th}} \text{ smallest} \\ 0, & \text{otherwise} \end{cases}$$

Runtime is proportional to # of comparisons:  $\leq C \sum_{i < j} X_{ij}$

**Expected runtime:**  $\leq E \left[ C \sum_{i < j} X_{ij} \right] = C \sum_{i < j} E[X_{ij}]$

$$= C \sum_{i < j} 1Pr[X_{ij} = 1] + 0Pr[X_{ij} = 0] = C \sum_{i < j} Pr[X_{ij} = 1]$$

Consider:



$S_{ij}$  will be partitioned after some run, say Run  $k$ . So the event of  $X_{ij}$  will either not happen or will happen at Run  $k$ , with the latter happen when either  $i$  or  $j$  is selected as the pivot. As number of elements in  $S_{ij}$  is less than or equal to the partition containing the  $i$  and  $j$  in Run  $k$ , we have:

$$Pr[X_{ij} = 1] = \sum_k Pr[k] Pr[X_{ij} = 1/k] \quad Pr[X_{ij} = 1, k] = \frac{Pr[k]}{P_1 \times P_2 \times \dots \times P_k} P_{1/k} \xrightarrow{P_{k1}}$$

$Pr[X_{ij} = 1] = P_{1/k} = \frac{2}{j-i+1}$ 
Probab of no breaking pivot selection in Run 1
Probab of  $i$  or  $j$  selected as pivot in Run  $k$

Let us proceed by considering the highest probability of  $X_{ij}$  :

$$C \sum_{i < j} Pr[X_{ij} = 1] = C \sum_{i < j} \frac{2}{j-i+1} = C \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} ; \text{ take } l = j-i$$

$$= C \sum_{i=1}^{n-1} \sum_{l=1}^{n-i} \frac{2}{l+1} < C \sum_{i=1}^{n-1} \sum_{l=1}^n \frac{2}{l} = 2C \sum_{i=1}^{n-1} C_1 \log n + C_2 \Rightarrow O(n \log n).$$

$n^{th}$  harmonic number  $\leftarrow \left[ \text{As } \sum_{l=1}^n \frac{1}{l} \propto \log n, n \rightarrow \infty \right]$

# Fast Fourier Transform (FFT)

## N-point Discrete Fourier Transform:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-\frac{2\pi i}{N} kn\right), k = 0, 1, \dots, N-1$$

$O(N^2)?$

$$\mathbf{F} \rightarrow F_{kn} = \exp\left(-\frac{2\pi i}{N} kn\right) \quad \mathbf{x} = [x_0, x_1, \dots, x_{N-1}]^T$$

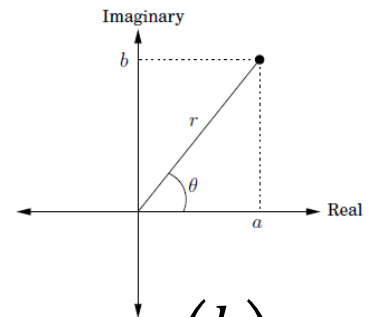
$$\mathbf{X} = \mathbf{F}\mathbf{x}$$

A complex number of unit magnitude

$$F_{kn} = \omega_N^{kn}, \omega_N = \exp\left(-\frac{2\pi i}{N}\right)$$

Complex numbers:

$$z = a + ib = r \exp(i\theta), r = \sqrt{a^2 + b^2} \text{ \& \; } \tan^{-1}\left(\frac{b}{a}\right)$$



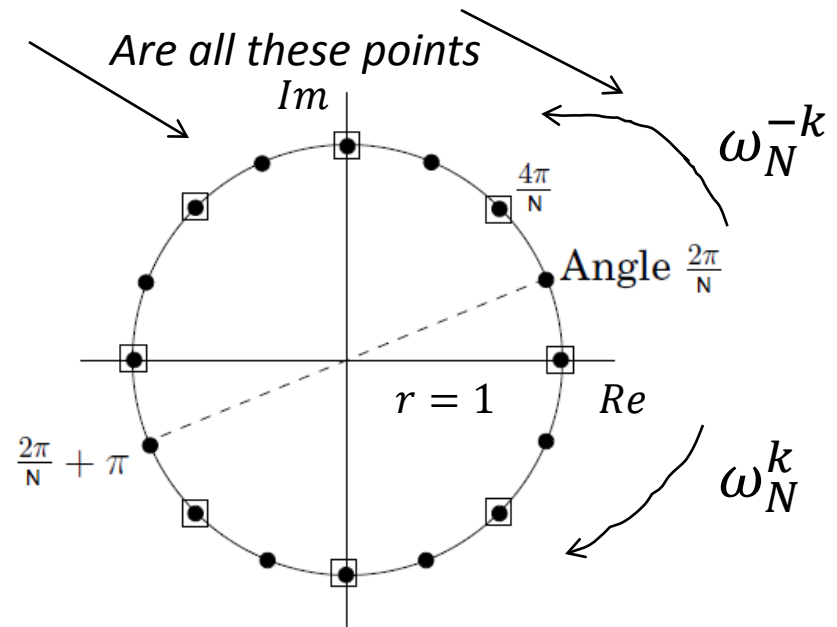
Consider:

$$\omega_N^k = \exp\left(-\frac{2\pi i}{N}k\right), k = 0, 1, \dots, N-1$$

$N^{\text{th}}$  roots of unity

$$N = 16$$

They are all complex numbers  
show in the complex plane!



Consider the above are the points where we want to evaluate the polynomials, for example,  $P$  with degree  $N-1$

Complex number

$$P(z) = p_0 + p_1 z + p_2 z^2 + \dots + p_{N-1} z^{N-1}$$

$$z \leftrightarrow \omega_N^k$$

$$= (p_0(z^2)^0 + p_2(z^2)^1 + p_4(z^2)^2 + \dots) + z(p_1(z^2)^0 + p_3(z^2)^1 + p_5(z^2)^2 + \dots)$$

$\downarrow$   
 $P_{\text{even}}$

$\downarrow$   
 $(z^2)^{\frac{N}{2}-1}$

$\downarrow$   
 $P_{\text{odd}}$

$\downarrow$   
 $(z^2)^{\frac{N}{2}-1}$

$$= P_{even}(z^2) + zP_{odd}(z^2) \rightarrow \text{This is kind of a divide!}$$

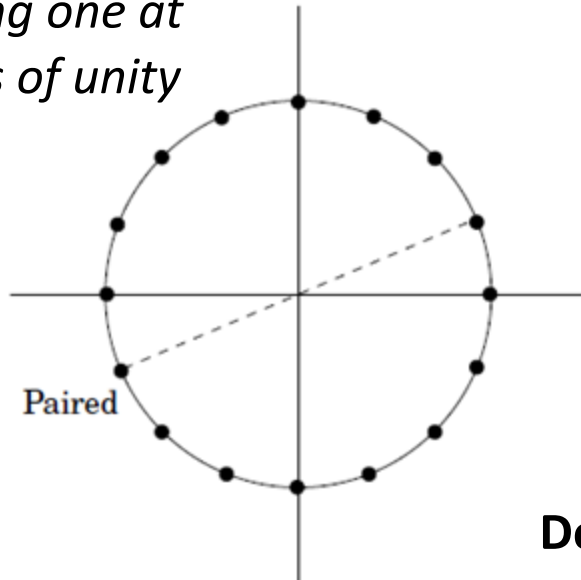
Now,  $P(\omega_N^k) = \sum_{n=0}^{N-1} p_n (\omega_N^k)^n, k = 0, 1, \dots, N-1$  DFT!

To get a  $P(\omega_N^k)$  we need to compute  $P_{even}((\omega_N^k)^2)$  and  $P_{odd}((\omega_N^k)^2)$

$\downarrow$   
N-1 degree, N parameters

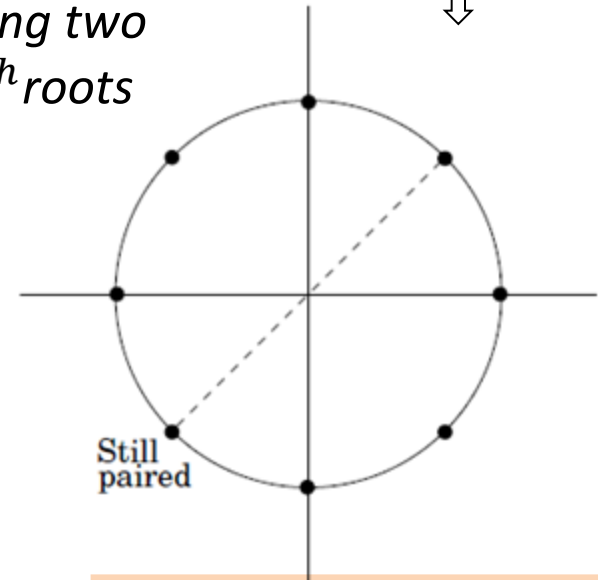
$\downarrow$   
(N/2)-1 degree, (N/2) parameters each

$\Downarrow$   
Evaluating one at  $N^{\text{th}}$  roots of unity



Divide and conquer  $\rightarrow$

$\Downarrow$   
Evaluating two at  $N/2^{\text{th}}$  roots of unity



Done? Only  $N/2$  points for  $P$ ?

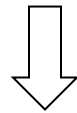
$$\omega_N^k, k = 0, 1, \dots, N-1$$

$$\left\{ \omega_{N/2}^k, k = 0, \dots, \frac{N}{2} - 1 \right\}$$

# The divide & conquer computation:

$$X_k = \sum_{n=0}^{N-1} x_n \omega_N^{kn}, k = 0, 1, 2, \dots, N-1$$

$$X_k = \sum_{q=0}^{N/2-1} x_{2q} \omega_{N/2}^{kq} + \omega_N^k \sum_{r=0}^{N/2-1} x_{(2r+1)} \omega_{N/2}^{kr}, k = 0, 1, \dots, N-1$$



$k = 0, 1, \dots, \frac{N}{2} - 1$  Enough!

$$X_k = \sum_{q=0}^{N/2-1} x_{2q} \omega_{N/2}^{kq} + \omega_N^k \sum_{r=0}^{N/2-1} x_{(2r+1)} \omega_{N/2}^{kr}, k = 0, 1, \dots, \frac{N}{2} - 1$$

$$X_k = \sum_{q=0}^{N/2-1} x_{2q} \omega_{N/2}^{lq} + \omega_N^k \sum_{r=0}^{N/2-1} x_{(2r+1)} \omega_{N/2}^{lr}, k = l + \frac{N}{2}, l = 0, 1, \dots, \frac{N}{2} - 1$$

$\swarrow$  same  $\searrow$  same  
 $\downarrow$   
 $-\omega_N^l$

So that's divide and conquer!

So the cost of evaluation:

$$T(N) = \begin{cases} 2T\left(\frac{N}{2}\right) + O(N), N > 1 \text{ (powers of 2)} \\ O(1), N = 1 \end{cases}$$

Master theorem:  $a = 2, b = 2, d = 1$

$$T(N) = O(N \log N)$$

That is:

$$\mathbf{Fx} \longrightarrow O(N \log N)$$

If you do not divide and conquer it is  $O(N^2)$ , the DFT.

If you do divide and conquer it becomes  $O(N \log N)$ , the FFT.

Application: we are going to use FFT for interpolation driven poly mult.

But before we do that...

## Inverse Discrete (Fast) Fourier Transform:

$$\mathbf{x} = \mathbf{F}^{-1} \mathbf{X}$$

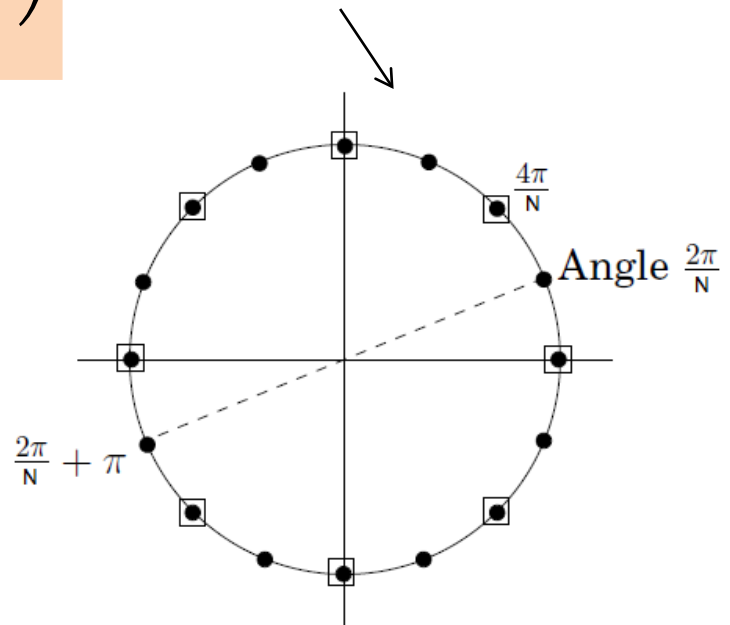
$$\mathbf{G} = \mathbf{F}^{-1}, G_{kn} = \frac{1}{N} F_{kn}^* = \frac{1}{N} \omega_N^{-kn} = \frac{1}{N} \exp\left(\frac{2\pi i}{N} kn\right)$$

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \exp\left(\frac{2\pi i}{N} kn\right), n = 0, 1, \dots, N-1$$

$$x_n^* = \frac{1}{N} \sum_{k=0}^{N-1} X_k^* \exp\left(-\frac{2\pi i}{N} kn\right)$$

So:

$$\mathbf{x} = \frac{1}{N} (\mathbf{F} \mathbf{X}^*)^*$$



N-point

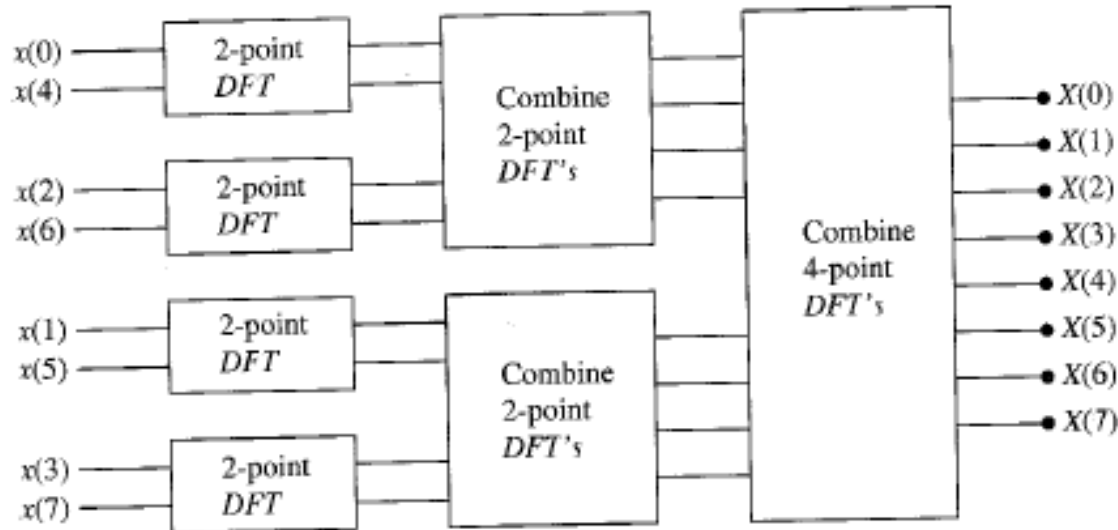
$$\begin{aligned}
 e_n &= x_{2n}, o_n = x_{2n+1} \\
 X_k &= E_k + \omega_N^k O_k; k = 0, 1, \dots, \frac{N}{2} - 1 \\
 X_{k+\frac{N}{2}} &= E_k - \omega_N^k O_k; k = 0, 1, \dots, \frac{N}{2} - 1
 \end{aligned}$$

 $E_k, O_k$ 

N/2-point

## Cooley–Tukey FFT algorithm

bit  
reversed  
order  
[N log(N)]



Only for iterative  
algo., not needed  
for recursive call.